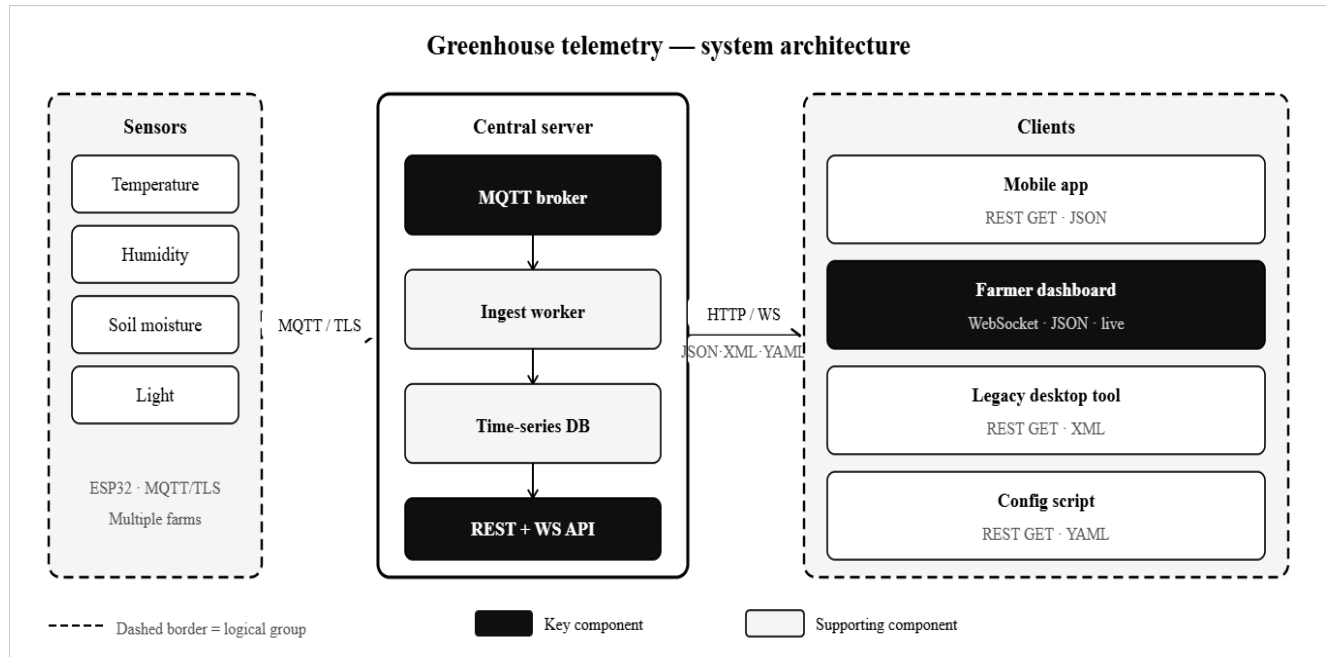


Planning & Architecture Document

a. System Architecture Diagram



b. Sequence Diagram

The sequence diagram shows the process of a soil moisture sensor sending data to the server and the dashboard receiving a live update instantly.

Scenario

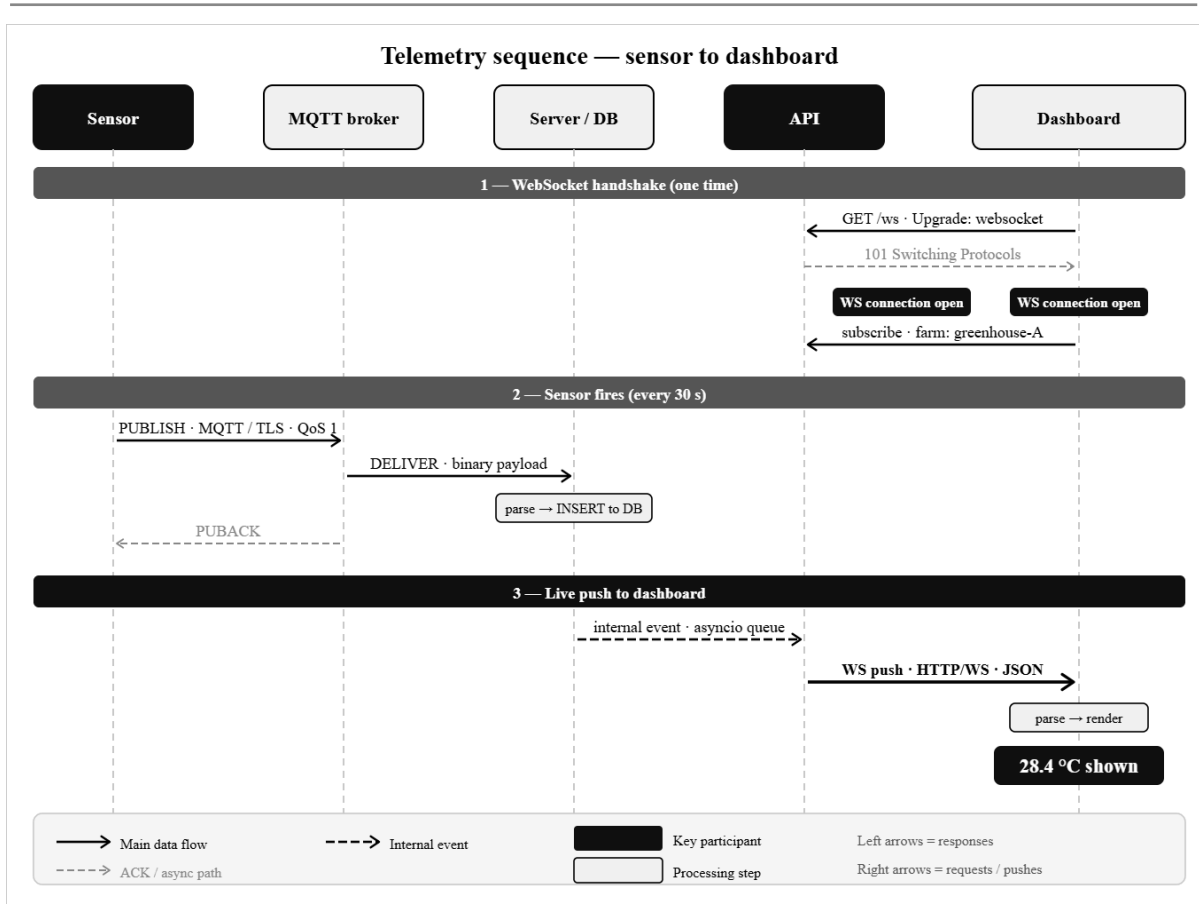
A soil moisture sensor in a greenhouse detects low moisture levels and sends the reading to the central server.

The process occurs as follows:

1. The sensor serializes the reading using Protobuf.
2. The reading is transmitted to the AsyncIO server.
3. The server stores the reading in the database.
4. The dashboard establishes a WebSocket connection.
5. The server pushes the latest reading to the dashboard in real time.

The WebSocket upgrade process works as follows:

1. The dashboard sends an HTTP request requesting a protocol upgrade.
2. The server responds with **101 Switching Protocols**.
3. A persistent WebSocket connection is established.
4. The server continuously pushes live sensor updates.



c. Data Model

The system stores information about sensors and sensor readings.

Sensor Entity

Field	Description
sensor_id	Unique sensor identifier
name	Sensor name
location	Farm or greenhouse location
type	Sensor type
status	Online or offline
created_at	Registration timestamp

Reading Entity

Field	Description
reading_id	Unique reading identifier
sensor_id	Related sensor
timestamp	Reading timestamp
temperature	Temperature value
humidity	Humidity value
soil_moisture	Soil moisture value

Protocol Buffers Schema

```
syntax = "proto3";
```

```
message Sensor {
  string sensor_id = 1;
  string name = 2;
  string location = 3;
  string type = 4;
}
```

```
message Reading {
  string reading_id = 1;
  string sensor_id = 2;
  string timestamp = 3;
  float temperature = 4;
  float humidity = 5;
  float soil_moisture = 6;
}
```

Protocol Buffers are used because they are compact, efficient, and faster than JSON for IoT telemetry communication.

d. REST Endpoint Specification

The REST API allows clients to retrieve historical telemetry data and manage sensors.

Endpoint	Method	Purpose
/sensors	GET	Retrieve all sensors
/sensors/{id}	GET	Retrieve a specific sensor

<code>/readings</code>	POST	Submit sensor readings
<code>/readings/latest</code>	GET	Retrieve latest readings

Example Endpoint

POST `/readings`

Request Body

```
{
  "sensor_id": "S01",
  "temperature": 28.5,
  "humidity": 70,
  "soil_moisture": 45
}
```

Response

```
{
  "message": "Reading stored successfully"
}
```

Status Codes

Code	Meaning
200	Request successful
400	Invalid request data
404	Sensor not found
500	Internal server error

e. YAML Configuration Schema

The YAML configuration file defines how farm sensors operate and connect to the telemetry server.

YAML Structure

```
sensor:
  id:
  type:
  location:
  sampling_interval:
```

server_url:

Example Configuration

```
sensor:
  id: S01
  type: soil_moisture
  location: Field_A
  sampling_interval: 30
  server_url: http://192.168.1.10:8000
```

Explanation

- `sampling_interval` defines how often readings are collected.
- `server_url` specifies the destination telemetry server.

YAML is used because it is human-readable and easy to configure.

f. Concurrency Model

The system uses AsyncIO to handle multiple farm sensors and dashboard clients concurrently without using threads.

Main Coroutines

Coroutine	Function
<code>receive_sensor_data()</code>	Receives telemetry readings
<code>store_readings()</code>	Stores readings in the database
<code>broadcast_updates()</code>	Sends live WebSocket updates
<code>handle_rest_requests()</code>	Processes REST API requests
<code>monitor_connections()</code>	Detects disconnected sensors

The system uses:

- AsyncIO event loop
- Non-blocking sockets
- Async queues

Slow Consumer Handling

If a dashboard client becomes slow:

- Messages are buffered temporarily
- Old messages may be dropped when queues become full
- Connections may be closed after timeout

This prevents one slow client from blocking the entire agricultural monitoring system.

g. Failure Handling

Sensor Disconnect

When a sensor disconnects:

- The server marks the sensor as offline
- The last known reading is retained
- Reconnection attempts are logged

Storage Failure

If the database fails:

- Readings are cached temporarily in memory
- Retry mechanisms are activated
- Alerts are generated for administrators

Server Overload

When too many sensors connect simultaneously:

- Rate limiting is applied
- Request queues are used
- Load balancing may distribute traffic
- Excess requests may receive:

503 Service Unavailable

The failure handling mechanisms improve reliability and ensure continuous monitoring of farming activities such as irrigation systems, greenhouse monitoring, livestock tracking, and crop storage monitoring.